Light

CPU Architecture & Von Neumann

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Purpose of the CPU
2. The fetch-execute cycle
3. CPU performance factors
Worked examples
Quick reference
Typical pitfalls

OCR GCSE Computer Science — **CPU Architecture (J277/01 Section 1.1)**. This topic covers how the Central Processing Unit works, including the Von Neumann architecture, the fetch-execute cycle, key components like the ALU and registers, and the factors that affect CPU performance.

Exam questions typically ask you to **describe** what happens when the CPU processes data, **identify** registers and their purposes, and **explain** how characteristics like clock speed, cache size and number of cores affect performance. Questions range from 1-mark definitions to 4-mark explanations requiring specific technical detail.

What you must know

This checklist covers the key exam-visible concepts. You should be able to explain each point and give an example where appropriate.

- **State** the purpose of the CPU and describe its role in a computer system.
- **Describe** the Von Neumann architecture, including how data and instructions are stored together in memory.
- **Identify** the function of each CPU component: ALU, Control Unit (CU), Cache, and Registers (MAR, MDR, PC, Accumulator).

- **Explain** the stages of the fetch-execute cycle and how data moves between components during each stage.
- **Explain** how clock speed, cache size and number of cores affect CPU performance.
- **Compare** the effects of upgrading different CPU characteristics on overall system performance.

> **Specification note.** OCR J277 Section 1.1 explicitly requires understanding of the purpose of the CPU, Von Neumann architecture, common CPU components and their functions, the fetch-execute cycle, and how CPU characteristics affect performance. Cache was added to the specification in the September 2024 update.

Toolkit

Von Neumann architecture — key principle

In Von Neumann architecture, **data and instructions are stored together in the same memory**. The CPU fetches instructions from memory, decodes them, and executes them one at a time in sequence. This is the basis of how almost all modern computers work.

The four key registers

Register	Full name	Purpose in the fetch-execute cycle
PC	Program Counter	Holds the **address** of the next instruction to be fetched from memory
MAR	Memory Address Register	Holds the **address** of the memory location being read from or written to
MDR	Memory Data Register	Holds the **data** or instruction that has been fetched from memory, or data to be written to memory
ACC	Accumulator	Holds the **results** of calculations performed by the ALU

CPU components — function summary

Component	Function
ALU (Arithmetic Logic Unit)	Performs all **arithmetic** (addition, subtraction, multiplication, division) and **logical** (comparisons, AND, OR, NOT) operations
CU (Control Unit)	**Controls** and **coordinates** the activities of all other components; decodes instructions; sends control signals

Component	Function
Cache	Small, very fast memory **inside the CPU** that stores frequently used data and instructions to speed up processing
Registers	Tiny, extremely fast memory locations **inside the CPU** that hold data, addresses and instructions currently being used

Fetch-execute cycle – the three stages

1. **FETCH:** The next instruction is fetched from memory and placed in the MDR. The PC is incremented.
2. **DECODE:** The Control Unit decodes the instruction to determine what operation needs to be performed.
3. **EXECUTE:** The instruction is carried out — this may involve the ALU performing a calculation, data being moved, or a value being stored.

CPU performance factors – the three characteristics

Characteristic	How it affects performance
Clock speed (measured in GHz)	Higher clock speed = more cycles per second = more instructions processed per second
Number of cores	More cores = more instructions can be processed **simultaneously** (in parallel)
Cache size	Larger cache = more frequently used data stored close to CPU = fewer slow accesses to RAM

Exam tip: When asked about registers, always state what the register **holds** (address, data, or result) and link it to a specific stage of the fetch-execute cycle. Generic answers like "stores information" will not gain marks.

1. Purpose of the CPU and Von Neumann architecture

The **Central Processing Unit (CPU)** is the "brain" of the computer. Its purpose is to **fetch data and instructions from memory**, **decode** those instructions, and then **execute** them. Every action a computer performs — from displaying text on screen to running complex calculations — is controlled by the CPU.

1.1 What the CPU does

The CPU carries out the instructions that make up computer programs. It processes input, performs calculations, makes decisions based on logic, and produces output.

Without the CPU, a computer would have no way of running software or responding to user actions.

Inside the CPU, instructions are processed one at a time in a continuous cycle called the **fetch-execute cycle** (also known as the fetch-decode-execute cycle). This cycle repeats billions of times per second in a modern processor.

1.2 Von Neumann architecture

Almost all modern computers are based on the **Von Neumann architecture**, named after mathematician John von Neumann who described it in 1945. The key features are:

- **Data and instructions are stored together** in the same memory (RAM).
- Instructions are **fetched from memory**, decoded, and executed **sequentially** (one after another).
- The CPU contains a **Control Unit**, an **ALU**, **registers**, and is connected to memory and input/output devices via **buses**.

This design means that a single computer can run different programs simply by loading different instructions into memory — the hardware does not need to change.

1.3 Main CPU components

The CPU contains several key components that work together:

Arithmetic Logic Unit (ALU): This performs all arithmetic operations (such as addition, subtraction, multiplication, division) and all logical operations (such as comparisons: equal to, greater than, less than; and Boolean operations: AND, OR, NOT).

Control Unit (CU): This coordinates and controls all the activities of the CPU and the wider computer system. It fetches instructions from memory, decodes them to work out what needs to be done, and sends control signals to other components to make it happen.

Cache: This is a small amount of very fast memory located inside the CPU. It stores copies of frequently used data and instructions so they can be accessed much faster than fetching them from RAM. Cache significantly speeds up processing.

Registers: These are tiny, extremely fast storage locations inside the CPU. They hold the data, addresses, and instructions that are currently being processed. Because they are inside the CPU, registers can be accessed almost instantly.

> **Exam tip:** A common 2-mark question is "Describe what happens in the CPU." A strong answer mentions that **data is fetched from memory** and then **processed**. Better answers mention specific components like the ALU performing calculations.

Worked Example 1 – Stating the purpose of the CPU

Question: State the purpose of the CPU. [2 marks]

> **Model answer:**
> The CPU fetches instructions/data from memory **(1)** and processes/executes them **(1)**.

> **Exam tip:** For 2-mark "state" questions, give two distinct points. "Fetches and

processes" covers both marks. Avoid vague phrases like "runs the computer".

Worked Example 2 — Describing Von Neumann architecture

Question: Describe two features of Von Neumann architecture. [2 marks]

Model answer:
1. Data and instructions are stored together in the same memory **(1)**.
2. Instructions are fetched and executed sequentially (one at a time) **(1)**.

Exam tip: The shared memory concept is the most important feature. If asked for just one feature, use this one.

Worked Example 3 — Identifying component functions

Question: State the function of the ALU. [1 mark]

Model answer:
The ALU performs arithmetic operations (e.g. addition) and logical operations (e.g. comparisons) **(1)**.

Exam tip: For ALU questions, mention **both** arithmetic AND logical. Saying only "does calculations" may not gain the mark as it misses the logical operations.

Worked Example 4 — Explaining the role of cache

Question: Explain why cache improves CPU performance. [2 marks]

Model answer:
Cache is faster than RAM **(1)**, so frequently used data/instructions can be accessed more quickly, reducing the time the CPU spends waiting for data **(1)**.

Exam tip: Always compare cache to RAM when explaining its benefit. The key point is that cache reduces waiting time because it is faster to access than main memory.

2. The fetch-execute cycle and registers

The **fetch-execute cycle** (also called the fetch-decode-execute cycle) is the process by which the CPU retrieves an instruction from memory, works out what it means, and then carries it out. This cycle repeats continuously while the computer is running.

2.1 The three stages explained

Stage 1: FETCH

The address of the next instruction is held in the **Program Counter (PC)**. This address is copied to the **Memory Address Register (MAR)**. The MAR sends the address to RAM, and the instruction stored at that address is fetched and placed into the **Memory Data Register (MDR)**. The PC is then incremented (increased by 1) so it points to the next instruction.

Stage 2: DECODE

The instruction in the MDR is passed to the **Control Unit (CU)**. The CU decodes the instruction — it works out what operation needs to be performed and what data (if any) is needed. The CU then prepares the appropriate components to execute the instruction.

Stage 3: EXECUTE

The instruction is carried out. This might involve:

- The **ALU** performing a calculation (e.g. adding two numbers)
- Data being read from or written to memory
- Data being sent to an output device
- The PC being changed to a different address (for a jump or branch instruction)

Results of ALU calculations are stored in the **Accumulator (ACC)**. The cycle then repeats from the fetch stage.

2.2 The four key registers

Registers are small, fast storage locations inside the CPU. Each has a specific purpose in the fetch-execute cycle:

Program Counter (PC): Holds the memory address of the **next** instruction to be fetched. After each fetch, the PC is incremented so it points to the following instruction. Branch instructions can change the PC to a different address.

Memory Address Register (MAR): Holds the address of the memory location that the CPU wants to read from or write to. When the CPU needs to fetch an instruction or data, the address is placed in the MAR and sent to RAM.

Memory Data Register (MDR): Holds the actual data or instruction that has been fetched from memory, or holds data that is about to be written to memory. It acts as a buffer between the CPU and RAM.

Accumulator (ACC): Holds the results of calculations performed by the ALU. If you add two numbers, the result is placed in the accumulator. It is also used to hold intermediate values during multi-step calculations.

> **Common misconception:** The Program Counter does NOT count how many programs have run, nor does it count instructions. It holds the **address** (memory location) of the next instruction. The name refers to its role in keeping track of where the program has reached.

Worked Example 5 — Describing the fetch-execute cycle

Question: Describe what happens during the fetch stage of the fetch-execute cycle. [3 marks]

Model answer:
The address of the next instruction is copied from the PC to the MAR **(1)**. The instruction at that address is fetched from RAM/memory and placed in the MDR **(1)**. The PC is incremented to point to the next instruction **(1)**.

Exam tip: For fetch-execute questions, use the register names (PC, MAR, MDR). Answers that just say "gets the instruction from memory" without mentioning specific registers typically score fewer marks.

Worked Example 6 — Identifying registers and their purpose

Question: A student says "The MAR holds the data that has been fetched from memory." Explain why this statement is incorrect. [2 marks]

Model answer:
The MAR holds the **address** of a memory location, not the data itself **(1)**. The data/instruction fetched from memory is held in the **MDR** (Memory Data Register) **(1)**.

Exam tip: MAR = Address, MDR = Data. If you remember nothing else about registers, remember this distinction. It is tested frequently.

Worked Example 7 — Role of the accumulator

Question: State the purpose of the accumulator in the CPU. [1 mark]

Model answer:
The accumulator holds the results of calculations performed by the ALU **(1)**.

Exam tip: Link the accumulator specifically to the ALU. Saying it "stores data" is too vague — it stores **results of calculations**.

Worked Example 8 — Multi-part register question

Question: A CPU is about to execute an instruction stored at memory address 1024. State the contents of the PC and MAR at the start of the fetch stage. [2 marks]

Model answer:
PC contains: 1024 (the address of the next instruction to fetch) **(1)**
MAR contains: 1024 (the address is copied from PC to MAR during fetch) **(1)**

Exam tip: In scenario questions, trace through the cycle step by step. At the **start** of fetch, PC holds the address and it gets copied to MAR. By the end of fetch, PC will have been incremented to 1025.

3. CPU performance factors

Three main characteristics affect how fast a CPU can process instructions: **clock speed**, **number of cores**, and **cache size**. Exam questions often ask you to explain how each affects performance, or to compare the effects of upgrading different characteristics.

3.1 Clock speed

The **clock speed** determines how many fetch-execute cycles the CPU can perform per second. It is measured in **hertz (Hz)**, with modern processors typically running at several **gigahertz (GHz)**. One GHz means one billion cycles per second.

A higher clock speed means more instructions can be processed each second, making the CPU faster. However, higher clock speeds also generate more heat and use more power, which is why there are practical limits to how fast a CPU can be clocked.

Example: A 3.0 GHz processor can perform 3 billion cycles per second. If each instruction takes one cycle, it could theoretically process 3 billion instructions per second (though in practice, many instructions take multiple cycles).

3.2 Number of cores

A **core** is an independent processing unit within a CPU. A **dual-core** processor has two cores, a **quad-core** has four, and so on. Each core can fetch, decode and execute instructions independently.

More cores allow the CPU to process multiple instructions **simultaneously** (in parallel), rather than one at a time. This is particularly beneficial for tasks that can be split into multiple independent parts, such as video editing, gaming, or running multiple applications at once.

However, not all software can take advantage of multiple cores. A program must be specifically designed for **parallel processing** to benefit from extra cores. A single-threaded program will only use one core regardless of how many are available.

3.3 Cache size

Cache is a small amount of very fast memory built into the CPU. It stores copies of data and instructions that are used frequently, so they can be accessed much faster than fetching them from RAM.

A larger cache means more frequently used data can be stored close to the CPU, reducing the number of times the CPU has to wait for slower RAM access. This is called improving the **cache hit rate**.

Cache is typically organised into levels: **L1 cache** is the smallest and fastest (closest to the cores), **L2** is larger but slightly slower, and **L3** is larger still but slower. All levels are still much faster than RAM.

3.4 Comparing the factors

When evaluating CPU upgrades, consider:

- **Clock speed** improves performance for all tasks but generates heat.
- **More cores** help with multi-tasking and parallel processing, but not single-threaded tasks.
- **Larger cache** reduces waiting time for memory access, benefiting most workloads.

The "best" upgrade depends on what the computer is used for. Gaming might benefit most from clock speed, while video editing benefits from more cores.

> **Exam tip:** When explaining performance factors, always state **how** the factor improves performance, not just that it does. "More cores are faster" scores less than "More cores allow multiple instructions to be processed simultaneously".

Worked Example 9 – Explaining clock speed

Question: Explain how increasing the clock speed of a CPU affects its performance. [2 marks]

Model answer:
A higher clock speed means more cycles/instructions can be processed per second **(1)**. This allows tasks to be completed faster **(1)**.

> **Exam tip:** Always link clock speed to "cycles per second" or "instructions per second". This shows you understand the mechanism, not just the outcome.

Worked Example 10 – Explaining multiple cores

Question: Explain why a quad-core processor may perform better than a dual-core processor for video editing. [3 marks]

Model answer:
A quad-core processor has four cores compared to two **(1)**. Each core can process instructions independently/simultaneously **(1)**. Video editing can be split into multiple tasks that run in parallel, so more cores allow more of the work to be done at the same time **(1)**.

> **Exam tip:** For "cores" questions, always mention that software must support parallel processing to benefit. This shows higher-level understanding and can gain additional marks.

Worked Example 11 – Comparing performance factors

Question: A user wants to upgrade their computer to run a video game faster. The game uses only one core. Explain which CPU upgrade would be most beneficial: increasing clock speed or adding more cores. [4 marks]

Model answer:
Increasing clock speed would be most beneficial **(1)**.
A higher clock speed means more instructions can be processed per second **(1)**.
Since the game only uses one core, adding more cores would not improve performance **(1)**.
The extra cores would sit idle as the game cannot make use of parallel processing **(1)**.

Exam tip: Extended questions often require you to justify your choice AND explain why the alternative is less suitable. Cover both sides for full marks.

Worked Example 12 — Full CPU performance comparison (8-mark style)

Question: A company is choosing between two CPUs for their office computers. CPU A has a clock speed of 4.0 GHz with 2 cores and 4MB cache. CPU B has a clock speed of 2.5 GHz with 8 cores and 16MB cache. Office workers mainly use word processing and spreadsheets, but some staff also edit videos. Discuss which CPU would be more suitable. [8 marks]

Plan:
- Analyse CPU A (high clock speed, fewer cores/cache)
- Analyse CPU B (lower clock speed, more cores and cache)
- Link to office work requirements (mainly single-threaded)
- Link to video editing requirements (benefits from multiple cores)
- Make recommendation with justification

Model answer:
CPU A has a higher clock speed of 4.0 GHz compared to CPU B's 2.5 GHz. This means CPU A can process more instructions per second, which benefits tasks like word processing and spreadsheets that typically run on a single core.
However, CPU B has significantly more cores (8 versus 2) and a larger cache (16MB versus 4MB). The additional cores allow multiple instructions to be processed simultaneously, which is beneficial for video editing as this task can be split across multiple cores for parallel processing.
The larger cache in CPU B means more frequently used data can be accessed quickly without waiting for RAM, which improves overall performance for all tasks.

For the majority of office workers doing word processing and spreadsheets, CPU A would be sufficient as these are typically single-threaded applications that benefit from high clock speed. However, for staff editing videos, CPU B would provide significantly better performance due to the ability to process multiple parts of the video simultaneously.
I would recommend CPU B for the office overall. While it has a lower clock speed, the 2.5 GHz is still adequate for office applications, and the extra cores and cache provide flexibility for video editing and running multiple applications simultaneously. The larger cache will also reduce waiting time for all users.

Exam tip: In discussion questions, always give a clear recommendation at the end with a summary of your reasons. According to the examiner's report, candidates who state "both are suitable" without making a final recommendation often lose marks.

Worked examples summary

The 12 worked examples above cover the main question types you will encounter on CPU architecture. Here is a summary of the key question patterns:

Example	Question type	Key skill
1	State the purpose (2 marks)	Two distinct points: fetch + process
2	Describe features (2 marks)	Von Neumann key features
3	State function (1 mark)	ALU: arithmetic AND logical
4	Explain benefit (2 marks)	Cache faster than RAM
5	Describe cycle stage (3 marks)	Use register names (PC, MAR, MDR)
6	Correct misconception (2 marks)	MAR = address, MDR = data
7	State purpose (1 mark)	Accumulator holds ALU results
8	Scenario application (2 marks)	Trace register contents through cycle
9	Explain factor (2 marks)	Clock speed = cycles per second
10	Explain with context (3 marks)	Cores + parallel processing
11	Compare and choose (4 marks)	Justify choice AND reject alternative
12	Discuss/evaluate (8 marks)	Analyse both options, recommend with reasons

Quick reference

Key terms — exam-ready definitions

Term	Definition (mark scheme style)
CPU	The Central Processing Unit; fetches instructions from memory and executes them
Von Neumann architecture	Computer design where data and instructions are stored together in the same memory and processed sequentially
ALU	Arithmetic Logic Unit; performs arithmetic operations (add, subtract, etc.) and logical operations (comparisons, AND, OR, NOT)
Control Unit (CU)	Controls and coordinates all CPU activities; decodes instructions and sends control signals
Cache	Small, fast memory inside the CPU that stores frequently used data to reduce access time
Register	Very small, very fast storage location inside the CPU for data currently being processed
Program Counter (PC)	Register that holds the address of the next instruction to be fetched
MAR	Memory Address Register; holds the address of the memory location being accessed
MDR	Memory Data Register; holds data fetched from memory or data to be written to memory
Accumulator (ACC)	Register that holds the results of calculations performed by the ALU
Fetch-execute cycle	The continuous process where the CPU fetches an instruction, decodes it, and executes it
Clock speed	The number of cycles the CPU performs per second, measured in Hz (usually GHz)
Core	An independent processing unit within a CPU; more cores allow parallel processing

Fetch-execute cycle — step by step

Stage	What happens	Registers involved
FETCH	Address copied from PC to MAR; instruction fetched from RAM to MDR; PC incremented	PC, MAR, MDR

Stage	What happens	Registers involved
DECODE	CU decodes instruction to determine what operation is needed	CU (uses instruction from MDR)
EXECUTE	Instruction carried out; ALU may perform calculations; results stored in ACC	ACC (for calculations)

CPU performance factors – comparison

Factor	How it improves performance	Limitation
Higher clock speed	More cycles per second = more instructions processed	Generates more heat; uses more power
More cores	Multiple instructions processed simultaneously (parallel)	Software must support parallel processing to benefit
Larger cache	More frequently used data stored close to CPU = faster access	Cache is expensive; diminishing returns beyond certain size

Typical pitfalls

Pitfall 1 — Confusing MAR and MDR.
Wrong: "The MAR holds the data fetched from memory."
Right: The MAR holds the **address**; the MDR holds the **data**. Remember: MAR = Address, MDR = Data.

Pitfall 2 — Misunderstanding the Program Counter.
Wrong: "The PC counts how many programs have run" or "The PC counts the instructions."
Right: The PC holds the **address** of the next instruction. It does not count anything — it stores a memory location.

Pitfall 3 — Forgetting to mention both parts of ALU function.
Wrong: "The ALU does calculations."
Right: The ALU performs **arithmetic operations** (add, subtract) AND **logical operations** (comparisons, AND, OR, NOT).

Pitfall 4 — Being vague about cache.
Wrong: "Cache makes the computer faster."
Right: Cache is **faster than RAM**, so frequently used data can be accessed more quickly, **reducing waiting time** for the CPU.

Pitfall 5 — Not using register names in cycle descriptions.

Wrong: "The instruction is fetched from memory."
Right: "The address is copied from the **PC** to the **MAR**; the instruction is fetched and placed in the **MDR**."

Pitfall 6 — Assuming more cores always means better performance.
Wrong: "8 cores is always better than 4 cores."
Right: More cores only help if the software supports **parallel processing**. Single-threaded programs only use one core.

Pitfall 7 — Confusing cache with secondary storage.
Wrong: "Cache stores files permanently."
Right: Cache is **volatile** memory inside the CPU for frequently used data. It loses contents when power is off and does not store files.

Pitfall 8 — Stating "information" instead of "data" or "instructions".
Wrong: "The CPU processes information."
Right: The CPU fetches **data and instructions** from memory and **processes** them. Be specific.

Pitfall 9 — Forgetting the accumulator's link to the ALU.
Wrong: "The accumulator stores data."
Right: The accumulator stores the **results of calculations performed by the ALU**.

Pitfall 10 — Not explaining HOW clock speed affects performance.
Wrong: "Higher clock speed is faster."
Right: Higher clock speed means more **cycles per second**, so more **instructions can be processed** each second.

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.